

IE3042 – SSS ASSIGNMENT



IT23545212

DENCIL P. A.N

Contents

1. <u>Task 1: Game Selection and Environment Setup</u>	3
1.1 Game Justification	3
1.2 Environment and Tools Setup	3
2. Task 2: Static and Dynamic Analysis	5
2.1 Static and Dynamic Disassembly	5
2.2 Register and Memory Identification	5
3. Task 3: Assembly-Level Modification	6
3.1 Instruction Identification	6
3.2 Modification Implementation (NOPing)	6
3.3 Permanent Binary Patching	6
4. Task 4: Security Analysis and Mitigation	8
4.1 Vulnerability Discussion	8
4.2 Proposed Modern Mitigation Strategies	8
5. Conclusion	9

1. Task 1: Game Selection and Environment Setup

1.1 Game Justification

For this project, **Cave Story (NXEngine-Evo v2.6.5-1)** was selected. Although the engine is a 64-bit refactor, it preserves the legacy logic of the original 2004 release. This selection is highly relevant to security because:

- **Lack of Modern Protections:** It lacks basic anti-debugging and integrity checks common in modern software.
- **Deterministic Memory:** It provides a clear demonstration of how user-mode applications manage critical state variables (Health) in unprotected RAM.
- **Legacy Logic:** It utilizes standard arithmetic assembly instructions (DEC, SUB) for state changes, providing an ideal attack surface for instruction patching.

1.2 Environment and Tools Setup

The analysis was conducted in a **Windows 11 Sandboxed Environment** to isolate the process. The primary tool utilized was **x64dbg**, an open-source binary debugger. x64dbg was chosen for its robust **TitanEngine** backend, which allows for real-time register tracking and the ability to apply permanent patches to the Portable Executable (PE) file.

Screenshot before :



2. Task 2: Static and Dynamic Analysis

2.1 Static and Dynamic Disassembly

Upon loading nx.exe into x64dbg, the entry point was identified. By performing a **Constant Search** for the player's initial health value (3 HP), the initialization function was located.

- **Key Function identified:** The health management routine responsible for collision-based state changes.

2.2 Register and Memory Identification

Through systematic debugging, the following was identified:

- **Base Pointer:** The **RCX register** acts as a pointer to the Player Object.
- **Memory Address:** The health variable is stored at a relative offset: $[rcx + 01]$.
- **Runtime Behavior:** When a collision occurs, the game fetches the value from this address, passes it to the CPU for decrementing, and writes it back.

3. Task 3: Assembly-Level Modification

3.1 Instruction Identification

The exact instruction responsible for health reduction was isolated at address **00007FF7DC72EF78**:

- **Original Instruction:** dec dword ptr ds:[rcx]
- **Opcode:** FF 09
- **Technical Justification:** This instruction directly decrements the 4-byte value at the address stored in RCX.

3.2 Modification Implementation (NOPing)

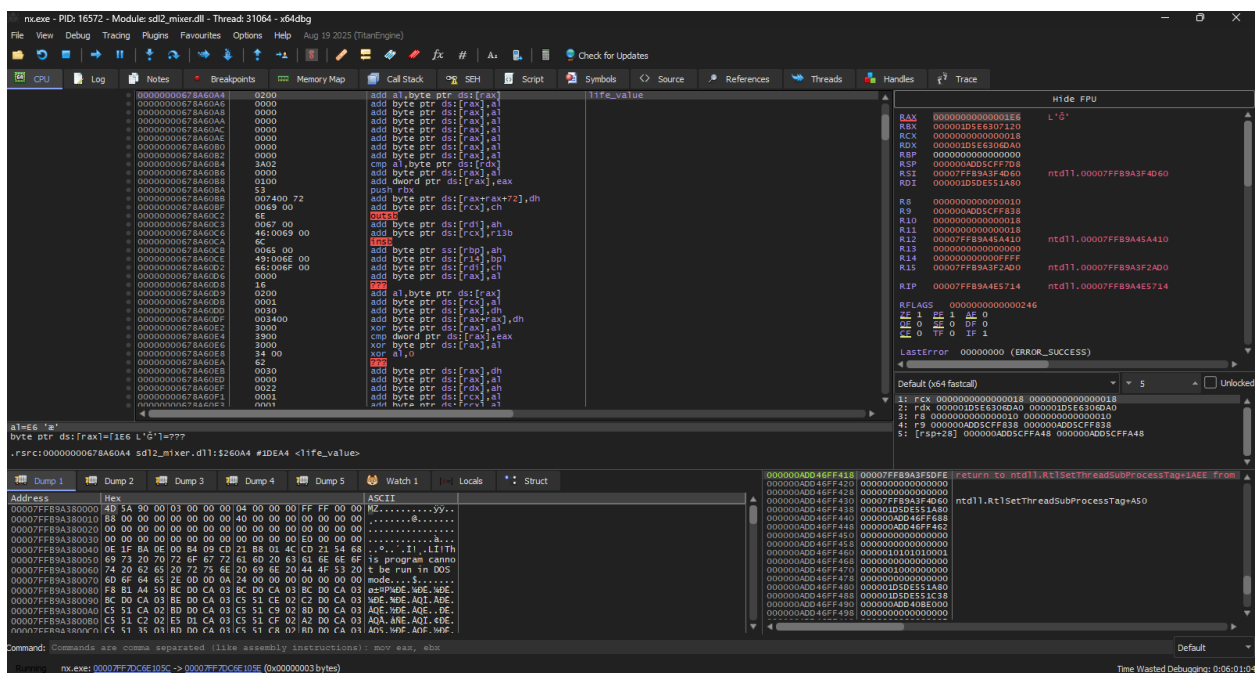
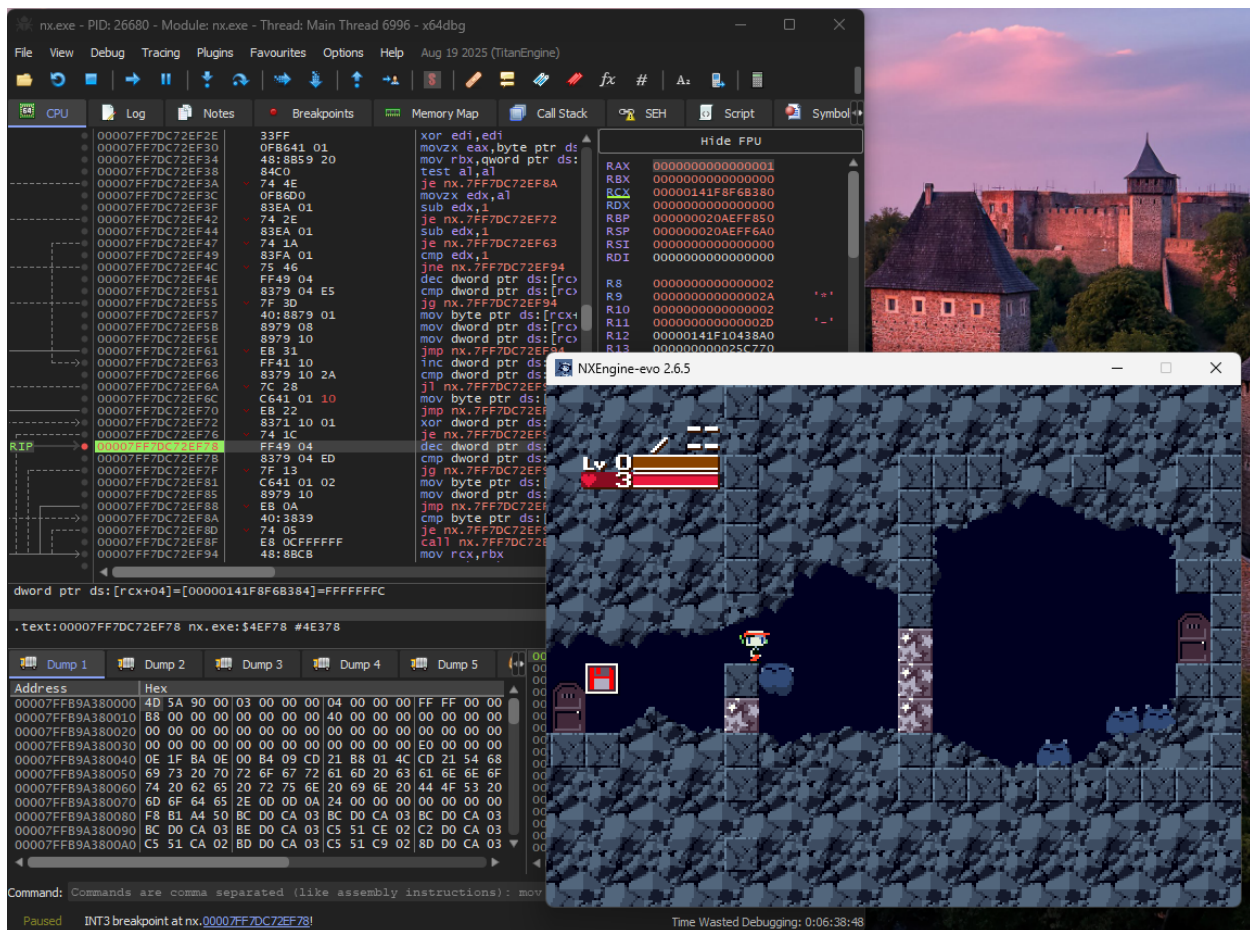
To neutralize this logic, a **NOP (No-Operation) Patch** was applied.

- **Action:** The DEC instruction was replaced with 90 90.
- **Before Modification:** Player health reduced from 3 to 2 upon enemy contact.
- **After Modification:** The CPU ignores the decrement logic. Health remains static at 3 HP regardless of collisions.

3.3 Permanent Binary Patching

Using the **Patches (Ctrl+P)** engine in x64dbg, the memory-resident changes were committed to a new executable: nx_modified.exe. This demonstrates the transition from a dynamic exploit to a permanent software modification.

Screenshot memory allocation and patching:



4. Task 4: Security Analysis and Mitigation

4.1 Vulnerability Discussion

This attack is possible due to several fundamental security flaws in legacy software architecture:

1. **Trusting the Client:** The game assumes that the local memory environment is "clean" and does not verify the integrity of its own instructions during runtime.
2. **Lack of Anti-Debugging:** The application does not call `IsDebuggerPresent()` or utilize `NtQueryInformationProcess`, allowing an attacker to hook the process and manipulate the EIP (Instruction Pointer).
3. **No Code Signing:** The OS executes the patched `nx_modified.exe` without verifying a digital signature, allowing unauthorized versions of the software to run.

4.2 Proposed Modern Mitigation Strategies

To prevent such manipulations in modern systems, I propose the following:

- **Anti-Debugging Traps:** Implement guard strips in the code that detect the presence of a debugger and terminate the process.
- **Checksum Verification (Self-Checksumming):** The application should perform a hash check of its `.text` section during startup. If the hash differs (e.g., due to NOPs), the program should refuse to load.
- **Address Space Layout Randomization (ASLR):** While present in modern OSs, making the health pointer dynamic or "obfuscated" via XOR operations would make memory scanning significantly harder.
- **Memory Protection:** Using **Data Execution Prevention (DEP)** and read-only memory segments for critical constants.

5. Conclusion

The project successfully demonstrated the reverse engineering lifecycle—from environment setup to the deployment of a modified binary. The vulnerability found highlights the critical need for **Integrity Checking** and **Anti-Tamper** mechanisms in modern software development.

Screenshot after :



VIDEO LINK : [IT23545212_SSS_Assignment.zip](#)

